



Общие постулаты для python-проектов

1. Мы всегда следуем пер8, если другого явно нигде не указано.
2. Используй редактор кода, который поддерживает работу с файлами .editorconfig
3. Единственное, с чем можно согрешить, и мы успешно грешим - это длина строк. Не надо ограничивать себя 79 символами, если хочется 119. 79 - это прошлый век и слишком сложно читается.
4. Пробелы вместо табов.
5. Мы используем уарф для форматирования.
6. Используй одинарные кавычки для строк, или двойные кавычки для строк с одинарными кавычками.
7. Для форматирования строк используем f-string. Если рядом с поправленным тобой кодом (или в нем) встретилась строка со старым форматированием (%/format), будь котиком, замени ее на f-string.
8. Сложные выражения в f строках не используем.
9. Для документации следуй PEP 257 и смотри, как написана текущая документация.
10. Для упорядочивания импортов мы используем isort с настройками из setup.cfg.
11. Используй абсолютные импорты.
12. Приводи в uppercase или lowercase перед сравнением строковые переменные справа и слева.
13. При передаче большого количества параметров именуй аргументы в kwargs стиле

Общие постулаты для любых проектов

1. Архитектура нового функционала или проекта никогда не является вопросом стиля или просто личным предпочтением. Иногда есть несколько допустимых вариантов. Она должна оцениваться при помощи данных, плюсов и минусов, либо на основе общепринятых инженерных принципов.
2. Ничто не оправдывает код, который определенно ухудшает общую работоспособность кода системы.
3. Прежде, чем добавлять функциональность, проверяй, не усложняет ли она код.
4. Используй линтеры в своих ветках, чтобы проверить стиль.
5. Мы не добавляем в .gitignore проекта пути, характерные для конкретной среды разработки/свои кастомные настройки разработки. Это следует убрать в глобальный gitignore.
6. Логи и комментарии пишем на русском языке.
7. Мы не пишем маленькие функции из одной строки. Сами по себе маленькие функции не являются чем-то плохим. Нет какого-то критерия на минимальный размер функции. Даже функции из одной строки имеют право на существование и при правильном применении бывают очень полезны. Вот основные моменты, когда выделение кода в функцию может ухудшить его:
 - Преждевременная оптимизация кода. На этапе написания кода не всегда понятно, каким он будет в финальном варианте. Бывает, что перед написанием функции разработчик хочет поместить в нее один функционал, но в итоге там остается другой. После написания кода можно сходить попить чаю и взглянуть на него еще раз перед отправкой на ревью. Может осталось что-то лишнее?
 - Злоупотребление DRY принципом. Тут все неоднозначно и сильно важен контекст. Главное не забывать об остальных принципах и не допускать перекося. Например, следование DRY в ущерб KISS.
 - Дублирование функций из готовых библиотек. Держим в голове готовые инструменты ([collections](#), [itertools](#), [functools](#) и [more-itertools](#)) и не ленимся читать документацию.
 - Соккрытие(без злого умысла) сложности. Самый простой способ сделать что-то проще - это разбить на маленькие и понятные кусочки. Чаще всего это действительно помогает и код становится легче читать, но иногда проблема требует более системного подхода и разбивать нужно уже классы, модули etc.



Юнит-тесты

1. Мы пишем тесты — это не просто слова.
2. Мы группируем тесты в классы или файлы по смыслу.
3. В классе теста мы соблюдаем порядок:
 - a. сначала setUp и tearDown
 - b. потом дополнительные функции
 - c. потом тесты
4. Каждый тест — атомарен. Роллбек транзакции происходит автоматически после каждого теста.
5. Мы не пишем тесты, зависимость друг от друга или тесты, чей порядок прохождения — важен.
6. Если тебе понадобился if в тесте, значит тебе нужен больше, чем один тест — по одному на каждую ветку условий if/else. Общую логику проверок/шагов теста можно объединить во вспомогательные функции.
7. Если мы написали код, который отправляет statsd-метрику, мы пишем тесты, которые проверяют отправку этой метрики.
8. Мы не меняем/добавляем дефолтные фикстуры ради 1 теста.
9. Если мы разрабатываем ручку api или cli-команду, то в unit-тестах мы обязательно пишем все проверки входных параметров.
10. Если у фреймворка (pytest) нет правил, в каком порядке сравнивать аргументы в assert, то мы используем вариант `assert actual == expected`

Требования к коммиту

1. Коммит должен быть маленьким.
Маленькие коммиты соответствуют парадигме юнит-тестирования и позволяют остановиться здесь и сейчас и продолжить работу потом другому человеку или в другом месте.
2. Коммит должен быть частым.
3. Коммит должен быть целостным.
То есть коммитятся сразу все связанные измененные файлы.
4. Семантически разные изменения (правка чужих опечаток, переносов строк и непосредственно работа по тикету) должны идти разными коммитами.
Да, это демотивирует делать исправления, но так диффы становятся в разы читабельнее.
5. Не должно быть коммитов типа "Коммит — просто коммит" или "Правки после ревью". Должно быть понятно, какого рода изменения были применены.
6. Коммиты должны отражать последовательность действий при работе над задачей.
7. Логические изменения должны быть сделаны в отдельных атомарных коммитах, код в которых желательно должен оставаться рабочим на любом участке ветки.
 - a. Тесты, фикстуры, документация атомарно сопровождают тестируемый код/функционал в рамках одного коммита
 - b. Правки тестов, кода, документации, PEP8, чего-либо ещё *стороннего функционала* (не связанных напрямую с задачей) должны выполняться в рамках отдельных коммитов
 - c. Исправления багов и улучшения производительности — это отдельные коммиты
 - d. Однотипное изменение в нескольких файлах/модулях можно поместить в один коммит

Подготовка задачи к код-ревью

Ответственность за поиск правильного решения задачи лежит на разработчике задачи. Это значит, что:

1. В мерж-реквесте не должно быть комментариев вроде "я сомневаюсь, что это хорошо" или "я не разобрался, поможешь?".
2. Все ожидаемые этапы pipeline проходят.



3. Ты проверил(а) задачу в тестовой среде.
4. Твой код соответствует требованиям code-style.
5. Все #TODO и #FIXME из твоего кода сразу переделались, как хорошо.